订单生命周期对标业界

从 3 态到 7 态：gomall 订单状态机的业务依据与实现 (v2.1)

gomall · 业务侧 deck

今日大纲

为什么 3 态不够用

业界对标：淘宝 / 京东主流 7 态

合法转换表：CanTransition 是单一权威

ShipOrder：状态机 + 事件 + 条件 UPDATE 三件套

ConfirmReceive：用户主动 + Cron 兜底

Refund 三态：业务对资金 / 评价 / 物流的强约束

RBAC：发货 / 同意 / 驳回挂 admin

兼容性：1/2/3 一字不改的迁移策略

业务承诺总表

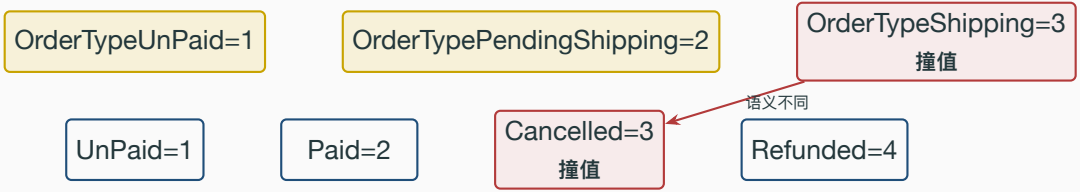
为什么 3 态不够用

deck 15 已经把”关单”讲透，但只覆盖了 1 个分支

- gomall 直到这一版之前只用了 3 个状态：**UnPaid / Paid / Cancelled**。
- deck 15 详细讲了 30min 关单链路——这是 `UnPaid → Cancelled` 这一条边。
- 真实电商订单的生命周期至少有 7 态：付款、发货、收货、退款各自是单独的合同节点。
- 缺位带来的业务后果：
  - **付款后无法跟踪履约**：商家发货 / 用户收货没有专门的状态。
  - **售后无法承接**：退款只能借用旧字段或脱离状态机硬改。
  - **数据指标失真**：成交转化、退款率、自动好评率全都没有口径。

deck 15 order-close-business 是本 deck 的前置；本 deck 在 v2.1 把状态机补齐到 7 态

旧 consts/order.go：两套常量并存的暗坑



- 旧文件并存两组常量，`OrderTypeShipping=3` 与 `Cancelled=3` **数值相同含义不同**。
- `OrderTypeShipping/Receipt` 从未被代码引用，撞值没爆雷，但是一颗定时炸弹。
- 这次合并把两套对齐：1= 待付 / 2= 已付待发 / 3= 已关闭，新增 4-7 接住业界 7 态。

consts/order.go:6–56；旧 *OrderTypeShipping* 数值由 *3* 调整为 *4* (未被引用，零回归风险)

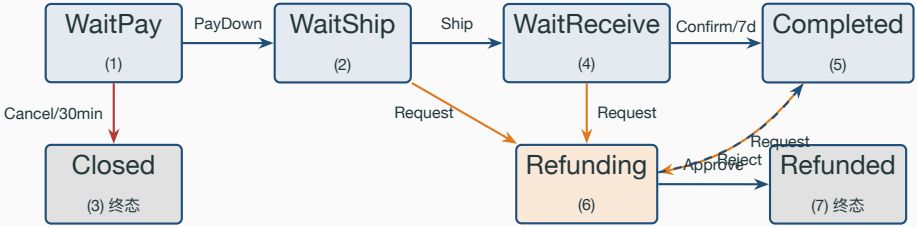
业界对标：淘宝 / 京东主流 7 态

业界对标：主流电商的 7 态怎么走

- 阿里 / 拼多多文档里把” 退款中” 和” 已退款” 列为独立态，是因为售后链路要冻结评价、扣物流费、对账等**独立业务规则**。
- 7 态版本是**电商业务和资金流的最低公约数**，做小了会回头补、做大了需要更复杂的子状态（如部分退款 / 部分发货）。

业界口径来源：阿里开放平台订单状态、京东商家中心订单状态、拼多多商家后台文档

gomall v2.1 完整状态机（7 节点 + 4 入口）



- **蓝色** = 正向流转；**橙色** = 退款触发；**灰色** = 终态。
- 三个终态：`Closed`（未付款关闭）、`Completed`（出 7d 退款窗口后）、`Refunded`（退款落地）。
- Refunding 是**唯一可双向出边**的节点：同意 → Refunded，驳回 → Completed。

service/order_state.go:18–26 转换表；*consts/order.go:6–15* 七个常量

合法转换表：CanTransition 是单一权威

合法转换表：一张 map 把规则写死

```go
20 var orderStateTransitions = map[uint][]uint{
21     consts.OrderWaitPay: {consts.OrderWaitShip, consts.OrderClosed},
22     consts.OrderWaitShip: {consts.OrderWaitReceive, consts.OrderRefunding},
23     consts.OrderWaitReceive: {consts.OrderCompleted, consts.OrderRefunding},
24     consts.OrderCompleted: {consts.OrderRefunding},
25     consts.OrderRefunding: {consts.OrderRefunded, consts.OrderCompleted},
26 }
```

Listing 1: service/order_state.go:20–26 转换表

- `key` 是 from，`value` 是允许的 to 集合——不在表里就是非法。
- `Closed` 和 `Refunded` 没有 key，`IsTerminalOrderState` 直接返回 true。
- **为什么放 service 不放 model**：状态机是业务规则，而不是数据约束；DB 只兜底”原子推进”，业务正确性靠这张表 + 测试。

service/order_state.go:20–26 表；:34–45 CanTransition；:56–59 IsTerminal

CanTransition：纯函数易于测试

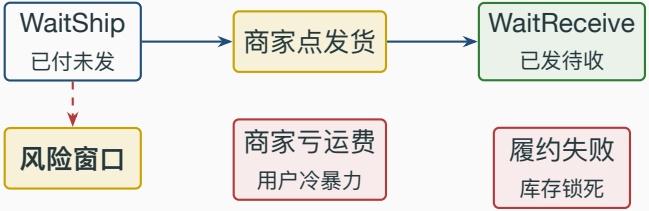
```go
34 func CanTransition(from, to uint) bool {
35    allowed, ok := orderStateTransitions[from]
36    if !ok {
37       return false
38    }
39    for _, s := range allowed {
40       if s == to {
41          return true
42       }
43    }
44    return false
45 }
```

Listing 2: service/order_state.go:34–45

- 纯函数，没有 DB / 没有 Redis / 没有 ctx ——**所有合法 + 非法组合都能用表驱动测试覆盖。**
- `order_state_test.go` 把 9 条合法、9 条非法、2 个终态、未知状态都跑一遍。
- 工业界做法：**业务规则放纯函数，副作用放 service**。单测可重复，无 flaky。

ShipOrder：状态机 + 事件 + 条件 UPDATE 三件套

发货为什么单独建一个状态



- **已付款 ≠ 已发货**：业务上要把两件事拆开，否则商家压货 / 用户催单都没数据抓手。
- 发货事件 `order.shipped` 会触发：物流系统对接、推送”已发货”、平台 7d 倒计时启动。
- gomall 当前不持久化物流单号（model 没字段），**tracking 仅经事件透传**；下游服务自行落表。

service/order_shipping.go:36–71 ShipOrder；events/events.go OrderShipped；

repository/db/model/order.go:7–18 model 当前无 tracking 字段

ShipOrder 实现：把规则、SQL、事件原子绑死

```go
36 func (s *ShippingSrv) ShipOrder(ctx context.Context, orderNum uint64, trackingNo, carrier string)
       error {
37    baseDao := dao.NewOrderDao(ctx)
38    order, err := baseDao.GetOrderByOrderNum(orderNum)
39    if err != nil { return err }
40    if order == nil || order.ID == 0 { return errors.New("订单不存在") }
41    if order.Type != consts.OrderWaitShip {
42       return ErrInvalidOrderStateTransition
43    }
44    return baseDao.DB.Transaction(func(tx *gorm.DB) error {
45       ok, err := dao.NewOrderDaoByDB(tx).ShipOrder(orderNum)
46       if err != nil { return err }
47       if !ok { return ErrInvalidOrderStateTransition }
48       return dao.NewOutboxDaoByDB(tx).Insert(
49          "order", "OrderShipped", "order.shipped", order.ID,
50          events.OrderShipped{...})
51    })
52 }
```

Listing 3: service/order_shipping.go:36–71 ShipOrder 主体

DAO 层条件 UPDATE：状态机的最后一道闸

```go
202 func (dao *OrderDao) ShipOrder(orderNum uint64) (bool, error) {
203     res := dao.DB.Model(&model.Order{}).
204         Where("order_num=? AND type=?", orderNum, consts.OrderWaitShip).
205         Update("type", consts.OrderWaitReceive)
206     if res.Error != nil { return false, res.Error }
207     return res.RowsAffected > 0, nil
208 }
```

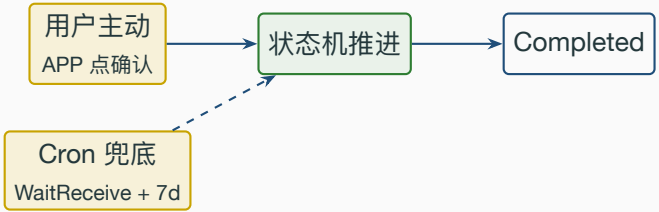
Listing 4: repository/db/dao/order.go:202–210 ShipOrder DAO

- `WHERE type=WaitShip` 兜住所有并发场景：第二次重发只会 `RowsAffected=0`。
- 这条 SQL = 状态机推进的**原子保证**，service 层就算被绕过也不会破坏数据一致性。
- 模式和 deck 15 讲的 `CloseOrderWithCheck` 一致——gomall 在每条边都重复这个模式。

repository/db/dao/order.go:202–210 ShipOrder；:214–222 ConfirmReceive；:240–248 ApproveRefund；:185–196 CloseOrderWithCheck 对照

ConfirmReceive：用户主动 + Cron 兜底

为什么 ConfirmReceive 必须双入口



- 现实中绝大多数用户**不主动确认**收货——收到货之后没人会主动点”已收到”。
- 业务承诺：**7 天兜底自动确认** = 资金结算给商家、平台抽佣结算、好评窗口开启。
- gomall 用 cron 每 6 小时扫 `WaitReceive + updated_at > 7d` 推进；这是**自动好评的资金触发器**。
- 同一条 SQL（`ConfirmReceive`）服务两个入口，靠 WHERE 兜底幂等。

*service/order_task.go:60–90 RunAutoConfirmReceive；initialize/cron.go 注册 0 0 */6*

RunAutoConfirmReceive：cron 兜底的标准模式

```go
60 func (s *OrderTaskService) RunAutoConfirmReceive() {
61    ctx := context.Background()
62    baseDao := dao.NewOrderDao(ctx)
63    orders, err := baseDao.GetTimeoutWaitReceive(autoConfirmReceiveDays, 100)
64    if err != nil {
65       util.LogrusObj.Errorf("Cron Job Error: fetch wait-receive orders failed: %v", err)
66       return
67    }
68    for _, order := range orders {
69       err := baseDao.DB.Transaction(func(tx *gorm.DB) error {
70          ok, err := dao.NewOrderDaoByDB(tx).ConfirmReceive(order.OrderNum)
71          if err != nil { return err }
72          if !ok { return errors.New("自动确认收货失败：订单状态已变更") }
73          return dao.NewOutboxDaoByDB(tx).Insert(
74             "order", "OrderCompleted", "order.completed", order.ID,
75             events.OrderCompletedEvent{..., Auto: true},
76          )
77       })
78       // ...
79    }
```

Refund 三态：业务对资金 / 评价 / 物流的强约束

Refunding / Refunded 为什么分两个态

- `Refunding`：用户已发起退款，但运营 / 商家**还没批**。
  - 评价系统：冻结评分，避免” 用户退款 □ 还给差评” 的尴尬。
  - 物流系统：如果还在 WaitShip 期间发起，商家不能再发货。
  - 客服系统：自动派单等待处理。
- `Refunded`：钱真退到用户账上 □ 才能触发资金扣回 / 库存对账。
- 业务上**发起退款 ≠ 退款成功**：拆开是为了让**每一步都有专门的下游消费者**。

> 把” 申请” 和” 完成” 拆开，是大多数金融型状态机的通用模式 (订单 / 提现 / 红包都一样)。

service/refund.go:46–91 RequestRefund + 93–128 ApproveRefund 是两条独立事件链

RequestRefund：from 集合是业务规则，不能写在 service 入口

```go
36 var refundAllowedFrom = []uint{
37    consts.OrderWaitShip,
38    consts.OrderWaitReceive,
39    consts.OrderCompleted,
40 }
41 // DAO 用 WHERE type IN (?) 一次拦截非法 from
42 func (dao *OrderDao) RequestRefund(orderNum uint64, allowedFrom []uint) (bool, error) {
43    res := dao.DB.Model(&model.Order{}).
44       Where("order_num=? AND type IN ?", orderNum, allowedFrom).
45       Update("type", consts.OrderRefunding)
46    return res.RowsAffected > 0, res.Error
47 }
```

Listing 6: service/refund.go:36–40 + DAO

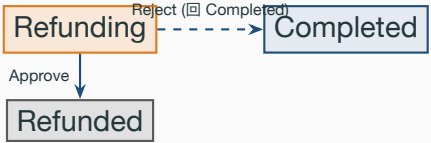
- 退款入口有**多个 from**（不同业务场景）——`WHERE type IN (?)` 把”读后写竞态”消灭在 SQL 层。
- 未付款 (`WaitPay`) 不在列表里：应走 `CancelUnpaidOrder` 关单链路，不是退款。
- `Closed` / `Refunded` 不在列表里：终态不可逆

ApproveRefund 本期只走状态机，不真退钱

- gomall v2.1 的 `ApproveRefund` **只推状态机** + 写 `order.refunded` 事件。
- 真退钱在下游：
  - **法币**：钱包服务消费事件 → 调用支付通道退款接口（占位 `tx_id`）。
  - **Web3**：listener 消费事件 → 链上转账后回写 `tx_id`。
- 这是**编排型 saga**：每一步都通过事件触发，避免在主链路里同步阻塞第三方。
- 路线图：等钱包服务落地后，把 `TxID` 字段填上，关闭"approve 后钱还没退" 的 gap。

service/refund.go:93–128 ApproveRefund；events/events.go OrderRefundedEvent.TxID 占位

RejectRefund：唯一允许” 回到 Completed” 的边

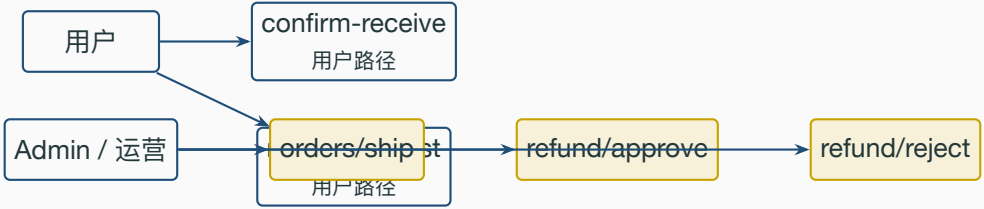


- Refunding → Completed 这条边**看上去像状态回退**，但实际上是业务上完全合法的” 售后未通过”。
- 业务后果：用户能再次发起退款（重新走一遍流程）；评价系统解冻；客服工单关闭。
- DAO 层依然用 `WHERE type=Refunding` 兜底，避免误把 WaitShip 或 WaitReceive 直接推到 Completed。

service/refund.go:130–157 RejectRefund；service/order_state.go:24 Refunding 出边集合

RBAC：发货 / 同意 / 驳回挂 admin

merchant 角色未落地，先挂 admin RBAC



- gomall 当前 RBAC 只有 `user` / `admin` 两角色；`merchant` 角色未落地。
- 发货 / 同意 / 驳回退款本应商家自己操作，**临时挂在 admin**（小工坊单老板 = admin OK）。
- 路线图：第一版商家后台落地后，把这三个接口挂回 `RequireRole("merchant", "admin")`。

routes/router.go 新增 6 个订单端点；middleware/rbac.go:48–86 RequireRole

router.go：6 个新接口的中间件链

```go
// 用户主动：确认收货 / 申请退款（幂等）
authed.POST("orders/confirm-receive", api.ConfirmReceiveHandler())
authed.POST("orders/refund/request", middleware.Idempotency(), api.RequestRefundHandler())
// 商家 / 运营：发货 / 同意 / 驳回退款。merchant 落地前先挂 admin RBAC
authed.POST("orders/ship", middleware.RequireRole("admin"), api.ShipOrderHandler())
authed.POST("orders/refund/approve", middleware.RequireRole("admin"), api.ApproveRefundHandler())
authed.POST("orders/refund/reject", middleware.RequireRole("admin"), api.RejectRefundHandler())
```

Listing 7: routes/router.go: 订单状态机扩展段

- `refund/request` 加幂等中间件：用户连点两次只生效一次。
- `confirm-receive` 不加幂等：DAO 层 `WHERE type=WaitReceive` 已经兜底。
- `ship` 不加幂等：商家系统多数已经在自家网关层做了。

routes/router.go 6 个新接口；middleware/idempotency.go 现有实现；middleware/rbac.go RequireRole

兼容性：1/2/3 一字不改的迁移策略

存量数据零变更：旧别名 Deprecated 兼容

- 关键约束：**DB 里数百万存量 `type` 不能动。**
- 1 = UnPaid → OrderWaitPay：语义**完全一致**，旧别名 `UnPaid` 保留兼容。
- 2 = Paid → OrderWaitShip：语义一致；旧别名 `Paid` 也保留。
- 3 = Cancelled → OrderClosed：语义一致（关单兜底）；旧名 `Cancelled` 保留。
- 撞值的 `OrderTypeShipping=3` 调到 4：**全代码 grep 0 引用**，安全。

> 靠” 旧别名做 const alias”，可以在不改 SQL / 不改前端字典的前提下把状态机一次到位。

consts/order.go:20–38 别名块；*payment.go:146* 仍用 *OrderTypePendingShipping* 不动

业务承诺总表

业务承诺总表 + 实现位置

业务承诺	状态边	实现位置
30min 未付自动关单	WaitPay → Closed	deck 15 已覆盖
商家发货后才计履约	WaitShip → WaitReceive	service/order_shipping.go:36
7d 自动确认收货	WaitReceive → Completed	service/order_task.go:60
用户可申请退款	* → Refunding	service/refund.go:46
售后通过才真退款	Refunding → Refunded	service/refund.go:93
售后驳回退回完成	Refunding → Completed	service/refund.go:130
终态不可逆	Closed / Refunded	service/order_state.go:20

7 条承诺 = 7 态状态机的核心；每条都对应一行 SQL + 一个 outbox 事件 + 一份单测。

service/order_state.go + order_shipping.go + refund.go + order_task.go 四件套

为什么”业务对标 + 状态机表 + 条件 UPDATE”是稳定范式

- **业务对标**（淘宝 / 京东 / 拼多多 7 态）——决定**有哪些节点**。
- **状态机表**（`orderStateTransitions` map）——决定**节点之间怎么连**。
- **条件 UPDATE**（`WHERE type=X`）——决定**并发下不会推错**。
- 三件套缺一不可：少了对标会做错；少了表会四处散落规则；少了 SQL 兜底会被并发打穿。

> 把”业务对标 □ 状态机表 □ 条件 UPDATE”做齐，订单系统就有**业务侧的不变量**，可以放心做扩展。

业务 Q&A（上）

Q1：为什么旧代码不直接重命名 `UnPaid → OrderWaitPay`？
A：怕一次改面太大引入 bug。先用 `const` 别名跑通编译 + 单测，老引用慢慢迁。

Q2：物流单号不入表，下游怎么消费？
A：`order.shipped` 事件里带 `tracking_no / carrier`，下游物流服务自行落表；outbox 模式标准操作。

Q3：7d 自动确认收货被用户投诉了怎么办？
A：状态机允许 `Completed → Refunding`，售后窗口期内仍可发起退款。Completed 不是死刑。

Q4：商家发货后用户不收货，能让商家主动确认吗？
A：不允许，会被薅羊毛（自买自卖确认后立即结算）。必须走 7d cron 兜底。

Q5：Refunding 状态商家先于运营批准发货了怎么办？
A：`ShipOrder` DAO 层 `WHERE type=WaitShip`，Refunding 不匹配 `RowsAffected=0`。商家发货被拒

业务 Q&A（下）

Q6：为什么 `ApproveRefund` 不真退钱？
A：主链路不阻塞第三方支付。状态推进 + outbox 后，钱包 / 支付通道服务异步消费事件真退钱并回填 `tx_id`。saga 风格。

Q7：cron 每 6 小时跑一次会不会太稀疏？
A：自动确认收货 = **资金结算触发器**，不要求秒级。6 小时一次足够；过于频繁反而徒增 DB 压力。

Q8：部分退款 / 部分发货什么时候支持？
A：等多 SKU 订单聚合根落地。届时 line item 表自己一套小状态机，聚合根 7 态保留不变。

Q9：3 态 → 7 态的迁移要不要发停机公告？
A：不用。存量 DB 数据完全兼容，常量层只是 alias；编译期就能验证全代码没改语义。

Q10：deck 15 讲的 30min 关单和本 deck 矛盾吗？
A：不矛盾。deck 15 是 `WaitPay` → `Closed` 这一条边的业务深度；本 deck 把整张图补

代码位置一览

状态机表 + 工具函数 *service/order_state.go:20–59*

7 态常量 + 旧别名 *consts/order.go:6–38*

OrderStateMap *consts/order.go:41–49*

ShipOrder service *service/order_shipping.go:36–71*

ConfirmReceive service *service/order_shipping.go:73–110*

RequestRefund service *service/refund.go:46–91*

ApproveRefund service *service/refund.go:93–128*

RejectRefund service *service/refund.go:130–157*

ShipOrder DAO *repository/db/dao/order.go:202–210*

ConfirmReceive DAO *repository/db/dao/order.go:214–222*

RequestRefund DAO *repository/db/dao/order.go:226–238*

GetTimeoutWaitReceive DAO *repository/db/dao/order.go:264–277*

Cron 自动确认收货 *service/order_task.go:60–90*

Cron 注册 (6h 一跑) *initialize/cron.go 0 0 */6 段*